Bahria University Lahore Campus

Department of Computer Science

Artificial Intelligence – BSCS 5

Assignment 2

UAV Fleet Planning with a Genetic Algorithm

Technical Report

Name: [Your Name]

Roll No: [Roll No]

1. Introduction

A company uses drones (UAVs) to survey a disaster area. We must decide where each drone is deployed, where it flies, how fast it flies and when (and how strongly) its sensor is switched on. A good plan covers the important areas and finds the targets, but it should also save battery, finish quickly and stay away from dangerous zones. These goals conflict, so there is no single correct answer. I solved this problem with a Genetic Algorithm (GA) written from scratch in Python.

The area is a 50 x 50 grid. It has 3 no-fly zones (blocked cells), 3 hazard zones (high wind / radiation), 3 priority (disaster) zones and 15 targets. 3 drones are launched from their deployment positions, visit 6 waypoints each and return to their deployment position. Each drone may fly at most 150 cells. One hazard zone lies on top of the most important disaster zone, so the GA must find a balance between coverage and safety.

2. Chromosome Design

Each chromosome is one complete plan for the whole fleet. It is a hybrid (real + binary) multi-segment encoding with one segment per drone: [deploy_x, deploy_y | (x, y, speed, power, on) x 6 waypoints].

Table 1: Genes of one drone segment (3 drones x (2 + 6 x 5) = 96 genes)

Gene	Type	Meaning	Range
deploy_x, deploy_y	real	deployment position (launch and landing point)	0 – 50
x, y	real	position of a waypoint	0 – 50
speed	real	speed on the way to this waypoint	1.0 – 3.0 cells/step
power	real	sensor power (radius 1 to 4 cells)	0 – 1
on	binary	sensor on/off on this leg (activation schedule)	0 or 1

Why this design? (1) It is simple: the genes are exactly the things the assignment asks us to decide (deployment positions, waypoints, speed, sensor power and sensor activation schedule). (2) Real numbers suit positions on a map: a small change in a gene gives a small change in the route, which works well with Gaussian mutation and blend crossover. (3) One segment per drone means a good route of one drone can be passed to a child as a whole block. (4) The on/off bit lets the GA switch the sensor off while travelling and save energy, and the power gene lets it choose a big or small sensor circle where it is on.

3. Fitness Function

F = 0.35·Coverage + 0.25·Detection − 0.15·Energy − 0.10·Time − 0.15·Hazard − 2·Penalty

- Coverage (maximise, weight 0.35): priority-weighted share of cells inside a sensor circle.
- Detection (maximise, weight 0.25): share of targets inside a sensor circle.
- Energy (minimise, weight 0.15): distance x (1 + 0.3·speed) + sensor power x time, divided by the maximum.
- Time (minimise, weight 0.10): flight time of the slowest drone / longest possible time.
- Hazard (minimise, weight 0.15): average hazard value under the flown path.

All objectives are scaled to 0–1, otherwise a large number (for example energy) would dominate. The two mission goals (coverage and detection) get 0.60 of the weight and the three costs get 0.40, so the drones are rewarded for doing the job and not for staying at the base. Coverage has the largest weight because surveying the disaster zones is the main purpose. Hazard and energy both get 0.15 because drone safety and battery are equally important. Time gets the smallest weight because the range limit already keeps the missions short.

Constraint handling uses both methods. Repair: if a deployment position or waypoint falls inside a no-fly zone, it is moved to the nearest open cell. Penalty: if a leg still flies through a no-fly zone, or a drone flies further than the maximum range, the fitness is reduced by 2 x (10 x share of the path inside no-fly zones + extra range). The penalty is large, so plans that break the rules cannot win.

4. Genetic Operators and Settings

Initial population: 50 random plans. Selection: tournament selection with k = 3 (pick k random plans, the best one becomes a parent); k controls the selection pressure. Crossover (rate 0.9): first a segment swap, where every drone's route is copied from parent 1 or parent 2; then a geometric blend, where one drone's positions, speeds and powers are a random mix of both parents (a point between them) and its on/off bits come from either parent. Mutation (rate 0.1): a waypoint moves by a small Gaussian step (about 3.0 cells) and its speed and sensor power change a little, a sensor on/off bit is flipped, a deployment position moves a little, and sometimes two waypoints swap their order. To keep diversity, the 2 best plans survive unchanged (elitism) and 2 new random plans are added every generation. Termination: 100 generations, or no improvement for 30 generations.

5. Results of the Main Run

With seed 42 the run reached 100 generations (4850 fitness evaluations). The best fitness improved from -2.409 in generation 0 to 0.391. The final plan covers 0.75 of the priority-weighted area, finds 100% of the targets, has energy 0.45, time 0.35, hazard 0.13 and penalty 0.00 (no rules broken). The average fitness stays below the best because two random immigrants (which usually break rules) enter every generation.

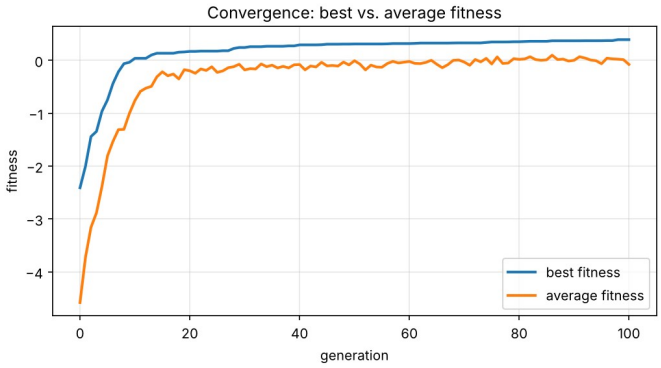


Figure 1: Best and average fitness over the generations

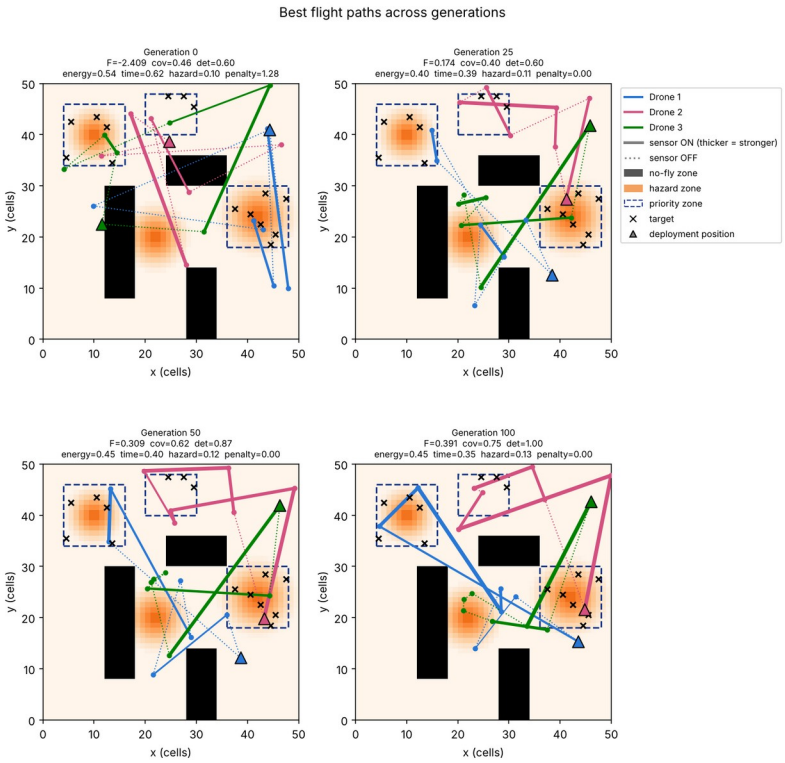


Figure 2: Best flight paths in generations 0, 25, 50 and 100 (triangle = deployment position, solid = sensor on, thicker = stronger, dotted = sensor off)

In generation 0 the routes are random zig-zags that cross the no-fly zones (black). Over the generations each drone is deployed close to a disaster zone, the routes go around the no-fly zones, the sensor is switched on (solid lines) inside the priority zones (blue boxes) and off (dotted lines) while travelling between them. The coverage heatmaps show the same: the covered area moves to the important zones.

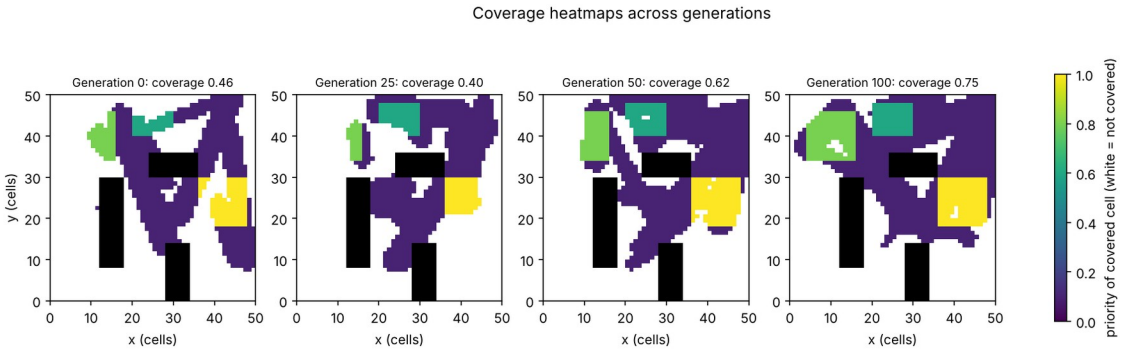


Figure 3: Coverage of the best plan across generations (colour = priority of the covered cell, white = not covered)

6. Hyperparameter Sensitivity Analysis

I changed one setting at a time and kept the others at their default values. Each setting was run with 3 seeds for 100 generations. Table 2 shows the final best fitness (mean ± standard deviation).

Table 2: Final best fitness for different settings (3 seeds)

Setting	Value 1	Value 2	Value 3
Population size	20: 0.338 ± 0.044	50: 0.396 ± 0.008	100: 0.415 ± 0.004
Mutation rate	0.02: 0.393 ± 0.002	0.1: 0.396 ± 0.008	0.3: 0.366 ± 0.012
Tournament k	2: 0.366 ± 0.039	3: 0.396 ± 0.008	5: 0.407 ± 0.004

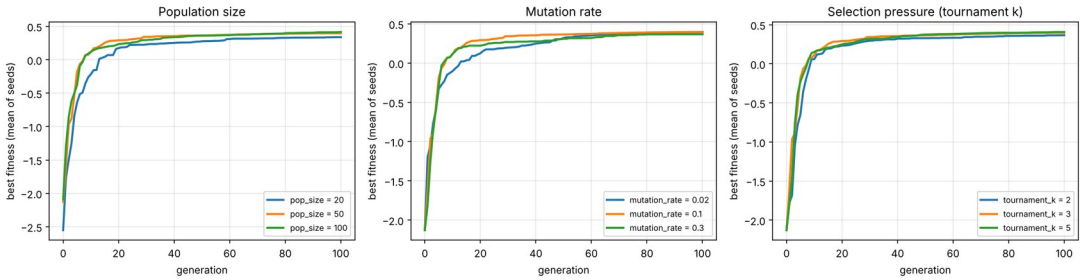


Figure 4: Mean best fitness for each setting

Population size: a bigger population gives better and more stable results, because it searches more of the map at the same time (more exploration). With 20 plans the results were worst and varied most between seeds. The cost is more fitness evaluations: 100 plans need twice the time of 50 plans.

Mutation rate: 0.02 and 0.1 gave almost the same result, but 0.3 was clearly worse. Too much mutation destroys good routes faster than selection can keep them, so the search becomes almost random (too much exploration). A small mutation rate is enough here because crossover and the random immigrants already keep the population diverse.

Selection pressure: a small tournament (k = 2) was the worst and the least stable, because weak plans are chosen as parents too often and the GA improves slowly. k = 5 was the best: good plans are chosen more often (more exploitation), and the immigrants and mutation still keep enough diversity to avoid getting stuck. This shows the balance between exploration (low pressure) and exploitation (high pressure).

7. GA versus Random Search

Random search creates random plans (with the same repair) and keeps the best one. To be fair it got exactly the same number of fitness evaluations as the GA (4850). Both were run with 5 seeds.

Table 3: GA vs. random search (mean ± standard deviation over 5 seeds)

Measure (best plan)	Genetic Algorithm	Random search
Fitness	0.374 ± 0.039	-0.859 ± 0.126
Coverage	0.707 ± 0.063	0.279 ± 0.140
Detection	0.947 ± 0.107	0.320 ± 0.176
Energy	0.367 ± 0.041	0.484 ± 0.032
Time	0.375 ± 0.052	0.560 ± 0.112
Hazard	0.115 ± 0.014	0.081 ± 0.033
Penalty	0.000 ± 0.000	0.448 ± 0.058

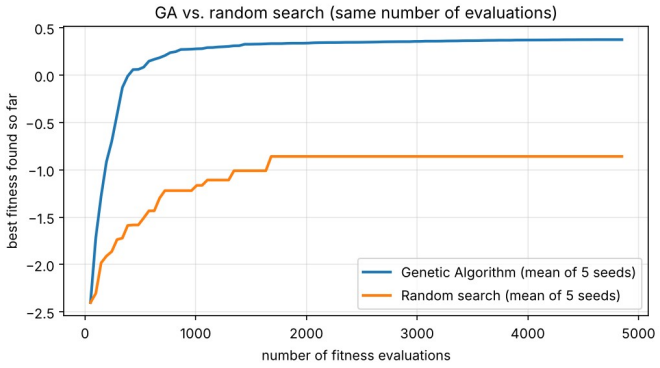


Figure 5: Best fitness found so far vs. number of evaluations

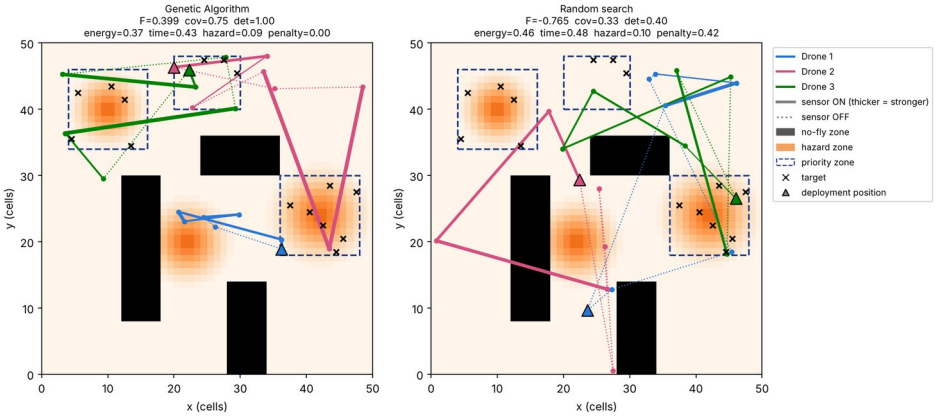


Figure 6: Best plan of the GA (left) and of random search (right), seed 1

The GA is much better than random search. Random search never found a plan that avoids all no-fly zones, so it always has a penalty; none of the 5 GA runs broke a rule. What makes the GA solution special: (1) structured paths: each drone is deployed near its own disaster zone and the routes go around the no-fly zones instead of random zig-zags; (2) sensor scheduling: the sensor is on over the priority zones and off while travelling, which saves energy; (3) priority focus: much higher coverage and (almost) all targets found; (4) less energy and a shorter mission time. The only cost where the GA is worse is hazard exposure: the most important disaster zone lies inside a hazard zone, and the GA accepts a little risk to survey it, which is exactly the trade-off the weights ask for. Random search only finds such a plan by luck, while the GA builds it step by step from good parts of earlier plans.

8. Conclusion and Limitations

I built a simple GA that plans the deployment positions, routes, speeds and sensor schedules of a drone fleet. The hybrid chromosome, the weighted fitness function, custom crossover and mutation, and repair plus penalty together give good and legal plans that are far better than random search. Limitations: the map is small and fixed, the energy and sensor models are simple, and the weighted sum gives only one compromise. In the future the GA could use a moving hazard field, more drones, and a multi-objective method such as NSGA-II to show many trade-offs at once.